Reviewer Pack — Minimal Rank of Geometric Group Representations and SAT Clau…

Entry be37b7e454f8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 11:26:41 UTC

Minimal Rank of Geometric Group Representations and SAT Clause Subset Entropy Inequality

Entry ID: `be37b7e454f8` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 11:26:41 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

> The minimal rank of a geometric group representation corresponding to a given Boolean satisfiability instance φ is linearly correlated with the entropy of its clause subsets, such that Entropy(φ) = Θ(Rank_G(G)), where Rank_G(G) is the minimal rank of the geometric group G associated with φ.

Rationale (proposer's reasoning):

> Geometric group theory provides a framework for studying discrete structures through group actions. This conjecture suggests that the complexity of Boolean satisfiability problems, particularly in terms of clause subset entropy, can be characterized by the algebraic structure of geometric groups. If true, it would establish a new connection between representation theory and computational complexity.

Taxonomy category: `GeometricGroupTheory_SATClauseSubsetEntropy` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `17fb42cb7bf67392`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 random seeds, the correlation coefficient between Entropy(φ) and Rank_G(G) is ≥ 0.7 for all φ with n ≤ 40 variables.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `geometric group theory AND satisfiability (SAT clause subset entropy inequality)` - `minimal rank geometric group representation AND Boolean satisfiability` - `entropy clause subsets AND geometric group theory representation SAT`

Top relevant hits considered: - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/2605.09347v1] Dsat: A Native SAT Solver for Discrete Logic - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/1802.04106v1] Recent progress in subset combinatorics of groups - [http://arxiv.org/abs/1807.11061v1] Clause Vivification by Unit Propagation in CDCL SAT Solvers - [http://arxiv.org/abs/1612.02639v3] Glider representations of group algebra filtrations of nilpotent groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def sat_clause_subset_entropy(phi):
        n = len(phi)
        total_clauses = 2 ** n
        subset_entropies = []

        for i in range(1, total_clauses):
            clause_subset = [phi[j] for j in range(n) if (i >> j) & 1]
            p = len(clause_subset) / total_clauses
            entropy_value = -p * math.log2(p) - (1 - p) * math.log2(1 - p)
            subset_entropies.append(entropy_value)

        return sum(subset_entropies) / len(subset_entropies)

    def geometric_group_rank(phi):
        n = len(phi)
        G = []
        for i in range(n):
```

```
            row = [0] * n
            row[i] = 1
            G.append(row)

        rank = 0
        for row in G:
            if any(x != 0 for x in row):
                rank += 1
                for j in range(len(G)):
                    if i != j and any(x != 0 for x in G[j]):
                        factor = G[j][i] / row[i]
                        for k in range(n):
                            G[j][k] -= factor * row[k]

        return rank

    def entropy(p):
        if p == 0 or p == 1:
            return 0
        else:
            return -p * math.log2(p) - (1 - p) * math.log2(1 - p)

    n_values = [5, 10, 15, 20, 30, 40]
    total_instances = 0
    entropy_sum = 0
    rank_sum = 0
    n_max = 0

    for n in n_values:
        for _ in range(5):  # Sample 5 instances per size
            phi = [random.choice([True, False]) for _ in range(n)]
            entropy_value = sat_clause_subset_entropy(phi)
            rank_value = geometric_group_rank(phi)

            total_instances += 1
            entropy_sum += entropy_value
            rank_sum += rank_value
            n_max = max(n_max, n)

    mean_entropy = entropy_sum / total_instances
    mean_rank = rank_sum / total_instances

    correlation_coefficient = (total_instances * sum(entropy_value * rank_value for entropy_value,
                               total_instances * mean_entropy * mean_rank) / \
                              math.sqrt((total_instances * sum(entropy_value ** 2 for entropy_val
                                        (total_instances * sum(rank_value ** 2 for rank_value in

    conjecture_holds = correlation_coefficient >= 0.7
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": total_instances,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
```

```
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27317297.py", line 103, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27317297.py", line 70, in run_trial
    rank_value = geometric_group_rank(phi)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_27317297.py", line 48, in geometric_gro
    factor = G[j][i] / row[i]
             ~~~~~~~~^~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution due to a division by zero error, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the division by zero error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12182
2	propose	ollama_remote	glm4:latest	0	0	13596
3	preregistration	ollama_remote	glm4:latest	0	0	9043
4	novelty	ollama_remote	glm4:latest	0	0	11106
5	novelty	ollama_remote	glm4:latest	0	0	9977
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42905
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13686
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10752
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13707
10	judge	ollama_remote	glm4:latest	0	0	14875

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 151828 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/be37b7e454f8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be37b7e454f8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/be37b7e454f8.tar.gz` (if generated)